

Laboratorio 2 – Periféricos de la Arquitectura AVR

Tecnologías de Microprocesamiento

Sergio Romero, Anthony Bonino, Santiago Moizo

Ingeniería Mecatrónica – UTEC

Semestre IV

Docentes: Jesús Martínez / Marcelo Díaz

Resumen—En este laboratorio se profundiza en el uso del microcontrolador ATmega328P programado en lenguaje ensamblador, incorporando periféricos y recursos de la arquitectura AVR. Las actividades propuestas comprenden el control de una matriz de LEDs con selección mediante UART, la generación de señales analógicas mediante una tabla LUT y un conversor DAC R–2R, el control de un plotter mediante comunicación serial, la comunicación USART entre dos microcontroladores y la automatización de una puerta de garaje mediante una máquina de estados y lectura por polling del sensor de obstáculos.

A lo largo del trabajo se aplican conceptos de manejo del Stack Pointer, direccionamiento de memoria, temporizadores y contadores, interrupciones, tablas de consulta, comunicación USART/UART y control de entradas y salidas digitales. Los programas se desarrollan en lenguaje ensamblador para el ATmega328P y se verifican mediante simulación y pruebas experimentales.

El equipo corresponde al Grupo 1. Para el Problema B se asignaron las señales 15 y 13, mientras que para el Problema C el personaje asignado al plotter es Porygon, número 137 de la Pokédex.

Palabras clave—ATmega328P, AVR, ensamblador, Stack Pointer, temporizadores, interrupciones, LUT, DAC R–2R, UART, USART, matriz de LEDs, plotter, máquina de estados.

I. INTRODUCCIÓN

El Laboratorio 2 continúa el trabajo realizado con el microcontrolador ATmega328P, incorporando recursos internos de la arquitectura AVR que permiten resolver aplicaciones de mayor complejidad.

Además del manejo directo de los puertos GPIO, en esta instancia se utilizan el Stack Pointer, el direccionamiento de memoria, temporizadores, contadores, interrupciones, tablas de consulta LUT y comunicación serie mediante USART o UART.

La metodología general consiste en analizar cada problema, identificar las entradas y salidas, definir la lógica de funcionamiento, implementar el código en lenguaje ensamblador, compilarlo, simularlo y finalmente comprobar su comportamiento sobre el montaje correspondiente. El desarrollo se versiona en GitHub para mantener trazabilidad de las modificaciones realizadas.

II. OBJETIVOS

II-A. Objetivo general

Profundizar en el uso del microcontrolador ATmega328P mediante la integración de periféricos y recursos de la ar-

quitectura AVR en aplicaciones prácticas programadas en lenguaje ensamblador.

II-B. Objetivos específicos

- Utilizar correctamente el Stack Pointer y el direccionamiento de memoria.
- Configurar y utilizar temporizadores y contadores del ATmega328P.
- Aplicar interrupciones para responder a eventos externos.
- Implementar tablas de consulta LUT para generar datos precalculados.
- Implementar comunicación serie mediante USART/UART.
- Controlar una matriz de LEDs mediante multiplexado.
- Generar señales analógicas mediante un DAC R–2R de 8 bits.
- Interconectar dos microcontroladores mediante comunicación serial.
- Diseñar máquinas de estados para sistemas secuenciales.
- Verificar los programas mediante simulación, medición y montaje físico.

III. MATERIALES

Los materiales y equipos utilizados durante el laboratorio son:

- Dos placas Arduino UNO con microcontrolador ATmega328P.
- Protoboard.
- Fuente de alimentación mediante USB o 5 V.
- Matriz de LEDs DOLANG ([Datasheet](#)).
- LEDs individuales.
- Resistencias.
- Pulsadores.
- DIP switch de al menos 3 posiciones.
- Cables de conexión.
- Resistencias para la red DAC R–2R.
- Osciloscopio.
- Conversor USB–Serial o monitor serial de la placa.
- Plotter del Laboratorio de Mecatrónica y su PLC de control.
- Relés electromecánicos o transistores MOSFET para la interfaz con el PLC.
- Sensores, alarma sonora y motor para la automatización de la puerta de garaje.

III-A. Hoja de datos del microcontrolador

Para la configuración de los registros, temporizadores, interrupciones, memoria y módulo USART se utiliza la hoja de datos del ATmega328P.

Acceso al Datasheet del ATmega328P

Figura 1. Enlace de acceso a la documentación utilizada durante el laboratorio.

IV. METODOLOGÍA GENERAL

Para cada actividad se sigue el siguiente procedimiento:

1. Análisis del enunciado y de los requisitos.
2. Identificación de entradas, salidas y periféricos utilizados.
3. Diseño del diagrama lógico o de la máquina de estados.
4. Asignación de registros y pines del ATmega328P.
5. Configuración de temporizadores, interrupciones y USART cuando corresponda.
6. Implementación del programa en lenguaje ensamblador.
7. Compilación del programa en Microchip Studio.
8. Simulación y verificación lógica del funcionamiento.
9. Implementación física y pruebas experimentales.
10. Registro de evidencias mediante capturas, mediciones y videos.
11. Incorporación en el informe de un acceso directo al video demostrativo de cada problema mediante Google Drive.
12. Versionado del código y registro de modificaciones significativas en GitHub.

V. PROBLEMA A: MATRIZ DE LEDs

V-A. Descripción

El Problema A consiste en programar el ATmega328P para controlar una matriz de LEDs y mostrar un mensaje que se desplace de derecha a izquierda. El texto debe ser legible, tener sentido completo y estar compuesto por al menos diez caracteres.

Además del mensaje desplazable, el sistema debe comunicarse mediante UART. Al iniciar la comunicación se debe presentar un mensaje de bienvenida y un menú que permita seleccionar entre el texto con desplazamiento y las figuras previamente implementadas para la matriz de LEDs.

V-B. Requisitos principales

- Mostrar una frase significativa de al menos diez caracteres.
- Desplazar el mensaje de derecha a izquierda de forma suave y continua.
- Permitir ajustar la velocidad de desplazamiento.
- Mantener el código organizado para facilitar cambios futuros.
- Implementar un mensaje de bienvenida mediante UART.
- Implementar un menú serial para seleccionar el mensaje o una figura.

V-C. Principio de funcionamiento de la matriz

La matriz se controla mediante multiplexado. En cada instante se activa una fila y se aplican sobre las columnas los bits correspondientes al patrón que se desea visualizar. A continuación se desactiva la fila actual y se activa la siguiente. Este procedimiento se repite con suficiente rapidez para producir la percepción de una imagen estable.

Para generar el desplazamiento del texto se almacenan los patrones de las letras y se modifica progresivamente la posición mostrada. El tiempo entre actualizaciones determina la velocidad visible del desplazamiento.

V-D. Asignación de pines

Para controlar las ocho filas y las ocho columnas de la matriz fue necesario distribuir las conexiones entre los puertos B, C y D del ATmega328P. Los pines PD0 y PD1 se reservaron para la comunicación UART, mientras que el resto de las salidas se utilizaron para realizar el multiplexado de la matriz.

La asignación utilizada en la implementación se muestra en la Tabla I.

Cuadro I
ASIGNACIÓN DE PINES DE LA MATRIZ DE LEDs

Función	Arduino UNO	ATmega328P
Filas de la matriz	D2–D7, A4–A5	PD2–PD7, PC4–PC5
Columnas de la matriz	D8–D13, A0–A1	PB0–PB5, PC0–PC1
UART RX	D0	PD0/RXD
UART TX	D1	PD1/TXD

Esta distribución permite utilizar las dieciséis conexiones necesarias para la matriz 8x8 y, al mismo tiempo, conservar PD0 y PD1 para la recepción y transmisión de datos mediante UART.

V-E. Mensaje y figuras implementadas

El mensaje utilizado en la implementación final es:

HELLO WORLD

Los patrones correspondientes a cada carácter se almacenan en la memoria de programa del ATmega328P. Cada letra se encuentra formada por diferentes columnas binarias que son incorporadas progresivamente al área visible de la matriz.

Además del mensaje desplazable, se implementaron tres figuras fijas: una sonrisa, un corazón y un asterisco, generadas en la evaluación continua. Estas figuras también se almacenan como patrones binarios de ocho bytes en la memoria de programa.

Las opciones disponibles mediante el menú UART se muestran en la Tabla II.

Cuadro II
OPCIONES DEL MENÚ UART DE LA MATRIZ

Opción	Acción
1	Mostrar figura Sonrisa
2	Mostrar figura Corazón
3	Mostrar figura Asterisco
4	Mensaje HELLO WORLD con desplazamiento
+	Aumentar velocidad de desplazamiento
-	Disminuir velocidad de desplazamiento

Al iniciar el programa, el modo seleccionado por defecto corresponde al mensaje desplazable. Mediante la terminal serial el usuario puede cambiar entre las diferentes figuras o regresar al mensaje sin necesidad de reiniciar el microcontrolador.

V-F. Comunicación UART

La comunicación UART se configuró a 9600 baudios utilizando 8 bits de datos, sin paridad y un bit de parada. Para una frecuencia de reloj de 16 MHz se utilizó el valor:

$$UBRR0 = 103 \quad (1)$$

correspondiente a la configuración utilizada por el programa para establecer la velocidad de comunicación.

Al iniciar el sistema, el ATmega328P transmite mediante UART el siguiente mensaje de bienvenida:

Bienvenido al Menu

Posteriormente se muestran las diferentes opciones disponibles para el usuario. Cada carácter recibido es analizado por la rutina de lectura UART y se modifica el modo de funcionamiento de la matriz según la opción seleccionada.

Cuando se recibe una opción válida, el microcontrolador envía además una confirmación por la terminal serial indicando la figura o el mensaje seleccionado.

V-G. Desplazamiento del mensaje

Para realizar el desplazamiento del mensaje se utiliza un buffer de ocho bytes denominado FRAME, almacenado en la memoria RAM del ATmega328P. Cada byte representa el patrón correspondiente a una de las filas visibles de la matriz.

Las columnas que forman el mensaje HELLO WORLD se encuentran almacenadas en memoria de programa mediante la tabla MENSAJE_COLS. El programa utiliza un índice para recorrer secuencialmente esta tabla.

En cada avance del desplazamiento se obtiene una nueva columna del mensaje y se actualizan los ocho bytes del buffer. El contenido existente se desplaza una posición y la nueva información se introduce desde el extremo derecho de la matriz.

De esta manera, los datos ingresan por la derecha y abandonan progresivamente la matriz por la izquierda, produciendo el desplazamiento requerido de derecha a izquierda.

Una vez alcanzado el final de la tabla del mensaje, el índice vuelve automáticamente al comienzo. Además, se incorporaron columnas en blanco al final del mensaje para permitir que el texto salga completamente de la matriz antes de comenzar nuevamente.

V-H. Control de velocidad

La velocidad de desplazamiento puede modificarse durante la ejecución utilizando los caracteres + y - recibidos mediante UART.

El programa utiliza una variable denominada `scrollldly`, cuyo valor determina la cantidad de ciclos de visualización que deben transcurrir antes de incorporar una nueva columna del mensaje.

El valor inicial utilizado es 15 y se establecieron los siguientes límites:

$$\text{SCROLL_MIN} = 5$$

$$\text{SCROLL_MAX} = 60$$

$$\text{SCROLL_STEP} = 5$$

Cuando se recibe el carácter +, el valor de `scrollldly` disminuye, por lo que se reduce el tiempo entre actualizaciones y el mensaje se desplaza más rápidamente.

Por el contrario, cuando se recibe el carácter -, el valor aumenta y el desplazamiento se vuelve más lento.

Los límites mínimo y máximo evitan que la velocidad alcance valores que dificulten la visualización del mensaje o provoquen valores fuera del rango establecido.

V-I. Código ensamblador

[Acceso al código ASM de la matriz de LEDs en GitHub](#)

Figura 2. Enlace al repositorio del código fuente del Problema A.

V-J. Simulación

Para comprobar el funcionamiento del programa antes de realizar la implementación física se desarrolló una simulación utilizando Proteus 8 Professional.

En el circuito simulado se incorporó una placa Arduino UNO con el microcontrolador ATmega328P, una matriz de LEDs 8x8 y una terminal virtual conectada a la interfaz UART.

La simulación permite verificar simultáneamente el multiplexado de la matriz, la visualización de las figuras, el desplazamiento del mensaje y la recepción de comandos mediante la terminal serial.

[Acceso a la simulación y captura de pantalla del circuito Problema A en GitHub](#)

V-K. Resultados y evidencia

Para verificar el funcionamiento del sistema se realizó inicialmente una simulación del circuito utilizando Proteus 8 Professional. En la simulación se implementó el ATmega328P mediante una placa Arduino UNO, una matriz de LEDs 8x8 y una terminal virtual para comprobar la comunicación mediante UART.

La matriz de LEDs fue conectada utilizando ocho líneas para las filas y ocho líneas para las columnas, de acuerdo con la distribución de pines definida anteriormente. Durante la simulación se comprobó el funcionamiento del multiplexado

de la matriz, realizando el barrido sucesivo de las ocho filas y aplicando en las columnas el patrón correspondiente a cada imagen.

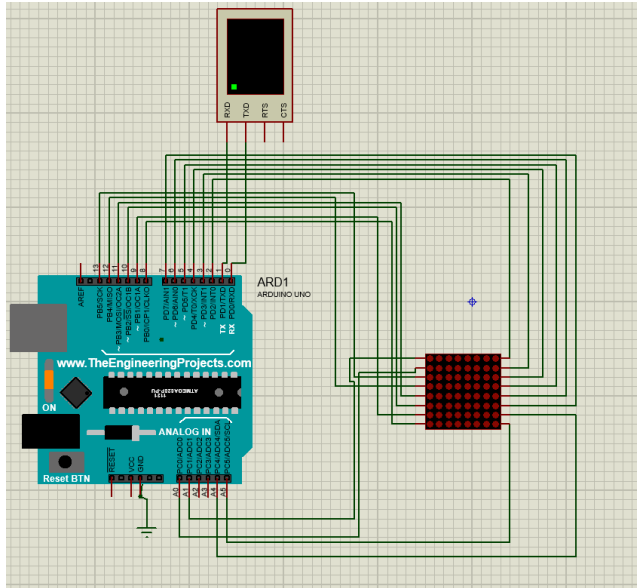


Figura 3. Simulación del circuito de la matriz de LEDs 8x8 en Proteus.

El menú implementado permite seleccionar las figuras Sonrisa, Corazón y Asterisco mediante las opciones 1, 2 y 3, respectivamente. Mediante la opción 4 se selecciona el mensaje HELLO WORLD con desplazamiento.

Cuando el usuario selecciona una de las tres primeras opciones, el microcontrolador carga desde la memoria de programa el patrón correspondiente a la figura seleccionada y realiza continuamente el barrido de las ocho filas de la matriz. De esta forma se comprobó la visualización de las tres figuras implementadas.

Para la opción 4, el sistema utiliza el buffer de ocho bytes almacenado en memoria RAM. Las columnas correspondientes al mensaje HELLO WORLD son incorporadas progresivamente al buffer. En cada actualización, los datos existentes se desplazan una posición y se introduce una nueva columna, generando visualmente el movimiento del texto de derecha a izquierda.

Se verificó el ajuste de la velocidad del desplazamiento mediante la terminal UART. Al ingresar el carácter +, se reduce el tiempo entre los avances del mensaje, produciendo un desplazamiento más rápido. Por el contrario, al ingresar el carácter -, se incrementa dicho tiempo y el mensaje se desplaza más lentamente.

Como evidencia adicional del funcionamiento experimental se incluye el video demostrativo del sistema, en el cual se observa la selección mediante UART, la representación de las diferentes figuras y el desplazamiento del mensaje sobre la matriz de LEDs.

[Acceso directo al video demostrativo en Google Drive](#)

VI. PROBLEMA B: DAC R-2R MEDIANTE LUT

VI-A. Descripción

El Problema B consiste en implementar un convertor digital-analógico (DAC) R-2R de 8 bits utilizando el microcontrolador ATmega328P de una placa Arduino Uno.

Las señales se almacenan en memoria de programa mediante tablas de consulta (LUT). Cada muestra de 8 bits es enviada periódicamente hacia las salidas digitales conectadas a la red R-2R, obteniéndose en su salida una tensión analógica proporcional al código digital aplicado.

Para el Grupo 1 fueron asignadas las señales número 13 y 15. La selección de la señal a generar se realiza mediante comunicación UART desde Microchip Studio utilizando Data Visualizer.

VI-B. Principio del DAC R-2R

El convertor R-2R utiliza solamente dos valores de resistencia, R y $2R$. La estructura permite ponderar cada uno de los bits del dato digital según su posición binaria.

Para un DAC ideal de 8 bits alimentado mediante una referencia V_{REF} , la tensión analógica de salida puede aproximarse mediante:

$$V_{OUT} = V_{REF} \frac{D}{256} \quad (2)$$

donde D representa el valor digital comprendido entre 0 y 255.

Por ejemplo, considerando una referencia próxima a 5 V:

$$D = 0 \Rightarrow V_{OUT} \approx 0 \text{ V} \quad (3)$$

$$D = 128 \Rightarrow V_{OUT} \approx 2.5 \text{ V} \quad (4)$$

$$D = 255 \Rightarrow V_{OUT} \approx 5 \text{ V} \quad (5)$$

VI-C. Cálculo y selección de las resistencias del DAC R-2R

Para implementar el convertor DAC R-2R se requieren únicamente dos valores de resistencia, denominados R y $2R$. La condición fundamental de la red es que el segundo valor sea exactamente el doble del primero:

$$2R = 2 \cdot R$$

Para el montaje experimental se seleccionó:

$$R = 10 \text{ k}\Omega$$

Por lo tanto:

$$2R = 2 \cdot 10 \text{ k}\Omega$$

$$2R = 20 \text{ k}\Omega$$

La relación obtenida entre ambos valores es:

$$\frac{2R}{R} = \frac{20 \text{ k}\Omega}{10 \text{ k}\Omega} = 2$$

por lo que se cumple correctamente la relación 2 : 1 requerida por la estructura del DAC R-2R.

La utilización de resistencias del orden de los kilohmios permite limitar la corriente demandada a las salidas digitales del ATmega328P durante la generación de las muestras de la LUT.

La salida del DAC fue conectada directamente al osciloscopio para observar las señales analógicas correspondientes a las señales 13 y 15.

VI-D. Asignación de pines

Debido a que los pines PD0 y PD1 se encuentran utilizados por la comunicación UART, las ocho salidas necesarias para el DAC fueron distribuidas entre los puertos B y D del ATmega328P.

Cuadro III
ASIGNACIÓN DE PINES PARA EL DAC R-2R

Bit DAC	ATmega328P	Arduino Uno
0 (LSB)	PB0	D8
1	PB1	D9
2	PB2	D10
3	PB3	D11
4	PB4	D12
5	PB5	D13
6	PD6	D6
7 (MSB)	PD7	D7

Los bits 0 a 5 son enviados mediante PORTB, mientras que los bits 6 y 7 son enviados mediante PORTD.

La rutina encargada de actualizar el DAC separa cada muestra mediante máscaras binarias. Para los seis bits menos significativos se utiliza:

```
dato AND 0x3F -> PORTB
```

mientras que para los dos bits más significativos se utiliza:

```
dato AND 0xC0 -> PORTD
```

De esta forma se mantiene el valor completo de 8 bits correspondiente a cada muestra de la LUT.

VI-E. Generación de señales mediante LUT

Las señales 13 y 15 se almacenaron como tablas de 256 muestras de 8 bits en la memoria de programa del ATmega328P.

El registro utilizado como índice de la tabla es de 8 bits, por lo cual recorre automáticamente:

$$0, 1, 2, \dots, 254, 255, 0, 1, \dots$$

Al producirse el desbordamiento desde 255 hasta 0 comienza nuevamente el recorrido de la señal, generando una forma de onda periódica sin necesidad de realizar una comparación adicional del final de la tabla.

La señal 13 está formada por valores que aumentan progresivamente desde 0x00 hasta aproximadamente 0xFF y posteriormente disminuyen nuevamente hasta 0x00, produciendo una forma de onda tipo campana.

La señal 15 contiene dos períodos completos dentro de las 256 muestras almacenadas. Por este motivo presenta aproximadamente el doble de frecuencia que la señal 13 para una misma frecuencia de muestreo.

VI-F. Configuración del Timer1

Para controlar el tiempo entre muestras se utilizó el Timer1 del ATmega328P en modo CTC.

La frecuencia de muestreo puede determinarse mediante:

$$f_s = \frac{f_{CPU}}{N(OCR1A + 1)} \quad (6)$$

donde:

$$f_{CPU} = 16 \text{ MHz}$$

$$N = 8$$

y el valor inicial utilizado fue:

$$OCR1A = 200$$

Por lo tanto:

$$f_s = \frac{16 \times 10^6}{8(200 + 1)} \quad (7)$$

obteniéndose:

$$f_s \approx 9950 \text{ Hz} \quad (8)$$

El período entre muestras resulta:

$$T_s = \frac{1}{f_s} \approx 100.5 \mu s \quad (9)$$

Por lo tanto, aproximadamente cada 100.5 μs se envía una nueva muestra hacia la red R-2R.

VI-G. Frecuencia teórica de las señales

En la señal 13, las 256 muestras almacenadas corresponden aproximadamente a un período completo. Su frecuencia teórica es entonces:

$$f_{13} = \frac{9950}{256} \approx 38.9 \text{ Hz} \quad (10)$$

y su período:

$$T_{13} = \frac{1}{38.9} \approx 25.7 \text{ ms} \quad (11)$$

En la señal 15 existen dos períodos dentro de las 256 muestras de la LUT. Por lo tanto, cada período está compuesto por aproximadamente 128 muestras:

$$f_{15} = \frac{9950}{128} \approx 77.7 \text{ Hz} \quad (12)$$

con un período:

$$T_{15} = \frac{1}{77.7} \approx 12.9 \text{ ms} \quad (13)$$

Se observa entonces que la frecuencia de la señal 15 es aproximadamente el doble que la correspondiente a la señal 13.

VI-H. Selección mediante UART

La comunicación UART fue configurada a 9600 baudios, utilizando 8 bits de datos, sin paridad y un bit de parada.

La selección de las señales se realizó mediante Data Visualizer de Microchip Studio conectado al puerto serie de la placa.

Cuadro IV
COMANDOS UART DEL GENERADOR DE SEÑALES

Comando	Acción
1	Seleccionar señal 13
2	Seleccionar señal 15
+	Aumentar frecuencia
-	Disminuir frecuencia

Al iniciar el programa se selecciona automáticamente la señal 13.

Al recibir el carácter 1, la dirección inicial utilizada para recorrer la LUT corresponde a la señal 13. Al recibir el carácter 2, dicha dirección se actualiza para comenzar a recorrer la tabla correspondiente a la señal 15.

Los caracteres + y - permiten modificar el valor de comparación del Timer1 y, por lo tanto, variar la frecuencia de muestreo y la frecuencia final de la señal generada.

VI-I. Diagrama del circuito

La red R-2R fue implementada con ocho entradas correspondientes a los ocho bits de salida del microcontrolador. La salida V_{OUT} se conectó al canal del osciloscopio y se utilizó una referencia de masa común entre la placa Arduino Uno, el DAC y el instrumento de medición.

[Acceso directo al circuito DAC R-2R](#)

Figura 4. Enlace al circuito DAC R-2R de 8 bits utilizado en el Problema B.

VI-J. Código ensamblador

El programa fue desarrollado íntegramente en lenguaje ensamblador para el ATmega328P. La generación periódica de muestras se realiza mediante una interrupción del Timer1, mientras que el programa principal permanece pendiente de los comandos recibidos mediante UART.

[Acceso al código ASM del DAC R-2R en GitHub](#)

Figura 5. Enlace al repositorio del código fuente del Problema B.

VI-K. Simulación

[Acceso a la simulación y capturas de resultados del Problema B en GitHub](#)

VI-L. Resultados experimentales

Luego de compilar y cargar el programa en el ATmega328P se verificó experimentalmente la generación de las señales mediante un osciloscopio digital.

Durante las mediciones se utilizó una escala temporal de aproximadamente 10 ms/div y una escala vertical de 2 V/div .

VI-L1. Señal 13: Al enviar mediante UART el comando 1, se seleccionó la señal 13. En el osciloscopio se obtuvo una forma de onda periódica tipo campana, presentando una subida progresiva desde aproximadamente 0 V hasta el máximo de salida del DAC, seguida por una disminución nuevamente hacia 0 V.

La forma obtenida experimentalmente coincide con la evolución de los valores almacenados en la LUT de la señal 13.

A partir de la escala temporal del osciloscopio puede observarse un período próximo a 25 ms , obteniéndose experimentalmente una frecuencia cercana a:

$$f_{13,exp} \approx 40\text{ Hz} \quad (14)$$

Este valor presenta una buena correspondencia con el valor teórico:

$$f_{13,teo} \approx 38.9\text{ Hz}$$

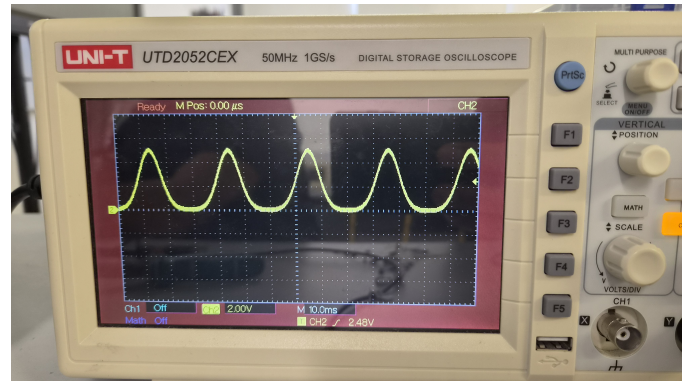


Figura 6. Señal 13 observada experimentalmente en el osciloscopio.

VI-L2. Señal 15: Posteriormente se transmitió mediante UART el comando 2. El microcontrolador cambió inmediatamente la LUT seleccionada y se observó en el osciloscopio la forma correspondiente a la señal 15.

En este caso se observa aproximadamente el doble de ciclos en el mismo intervalo temporal en comparación con la señal 13.

A partir de la escala del osciloscopio puede estimarse un período cercano a 13 ms , correspondiente a una frecuencia experimental aproximada de:

$$f_{15,exp} \approx 78\text{ Hz} \quad (15)$$

Este resultado coincide adecuadamente con el valor calculado de:

$$f_{15,teo} \approx 77.7 \text{ Hz}$$

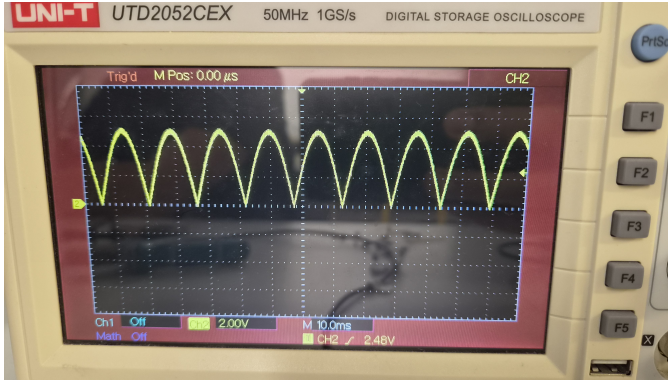


Figura 7. Señal 15 observada experimentalmente en el osciloscopio.

VI-M. Comparación de resultados

Cuadro V
COMPARACIÓN DE FRECUENCIAS TEÓRICAS Y EXPERIMENTALES

Señal	Teórica	Experimental
13	38.9 Hz	$\approx 40 \text{ Hz}$
15	77.7 Hz	$\approx 78 \text{ Hz}$

Los resultados experimentales presentan una buena correspondencia con los valores calculados teóricamente.

Se verificó además que el cambio entre las señales 13 y 15 se realiza correctamente mediante UART, observándose inmediatamente la modificación de la forma de onda en el osciloscopio.

VI-N. Video demostrativo

Como evidencia del funcionamiento experimental del DAC R-2R se incorpora el acceso directo al video demostrativo almacenado en Google Drive.

[Acceso directo al video demostrativo en Google Drive](#)

VI-Ñ. Conclusiones del Problema B

Se logró implementar correctamente un DAC R-2R de 8 bits controlado por un ATmega328P. Las muestras correspondientes a las señales 13 y 15 fueron almacenadas mediante tablas LUT y enviadas periódicamente al convertor utilizando interrupciones del Timer1.

La comunicación UART permitió seleccionar en tiempo real la señal a generar y modificar su frecuencia mediante comandos enviados desde Microchip Studio.

Las formas de onda obtenidas experimentalmente coinciden con el comportamiento esperado de las LUT implementadas. La señal 13 presentó una frecuencia cercana a 40 Hz, mientras que la señal 15 presentó una frecuencia cercana a 78 Hz, aproximadamente el doble de la primera.

De esta manera se comprobó el funcionamiento conjunto del temporizador, las tablas LUT, la comunicación UART y el convertor R-2R.

VII. PROBLEMA C: CONTROL DE PLOTTER

VII-A. Descripción

El Problema C consiste en programar un microcontrolador ATmega328P en lenguaje ensamblador para controlar el plotter disponible en el Laboratorio de Mecatrónica.

La selección de las figuras se realiza mediante comunicación serial UART. Desde una computadora el usuario envía un carácter al microcontrolador y, dependiendo de la opción seleccionada, el programa ejecuta automáticamente la secuencia de movimientos correspondiente.

La versión final permite dibujar individualmente un triángulo, un círculo, un pentagrama, un cubo 3D y una representación geométrica de Porygon. Además, mediante el comando T se dibujan las cinco figuras juntas en una zona reservada de la hoja A4.

Durante el desarrollo fue necesario realizar varias etapas de calibración para ajustar el tamaño de los dibujos, la separación entre ellos, el retorno del cabezal a la posición inicial y el comportamiento del sistema frente a los reinicios producidos durante el accionamiento del solenoide.

VII-B. Arquitectura del sistema

El funcionamiento general del sistema puede representarse mediante la siguiente cadena de control:

PC → UART → ATmega328P
→ Relés → PLC → Plotter

La computadora se utiliza para seleccionar la figura mediante un terminal serial. El ATmega328P recibe el carácter correspondiente y activa las salidas D2-D7 según la secuencia almacenada en el programa.

Las señales pasan por la etapa de interfaz de relés instalada en la estación y posteriormente llegan a las entradas digitales del PLC. Finalmente, el PLC controla los movimientos mecánicos y el accionamiento del solenoide del trazador.

VII-C. Conexiones con el plotter

Cuadro VI
CONEXIONES DEL ATMEGA328P CON EL SISTEMA DEL PLOTTER

Arduino	ATmega328P	Función
D2	PD2	Bajar solenoide – X0
D3	PD3	Subir solenoide – X1
D4	PD4	Movimiento abajo – X5
D5	PD5	Movimiento arriba – X6
D6	PD6	Movimiento izquierda – X7
D7	PD7	Movimiento derecha – X10

Los pines D0 y D1 se reservan para la comunicación UART:

D0 = PD0 = RX
D1 = PD1 = TX

VII-D. Comunicación UART

La comunicación serial se configuró a 9600 baud, 8 bits de datos, sin paridad y con 1 bit de parada.

El menú final implementado es:

Dibujos PLOTTER

```
1 Triangulo  2 Circulo  3 Pentagrama
4 Cubo 3D   P Porygon  T Todas
```

Cuadro VII
COMANDOS IMPLEMENTADOS

Comando	Acción
1	Dibujar triángulo
2	Dibujar círculo
3	Dibujar pentagrama
4	Dibujar cubo 3D
P / p	Dibujar Porygon
T / t	Dibujar todas las figuras

Los caracteres de retorno de carro y salto de línea recibidos desde el terminal son ignorados para evitar que sean interpretados como comandos.

También se reorganizaron las cadenas definidas mediante .db. El byte 0 se coloca únicamente al final del mensaje completo, evitando que la transmisión UART termine antes de mostrar la segunda línea del menú.

VII-E. Distribución de los dibujos en la hoja A4

Para obtener una distribución ordenada se dividió lógicamente el área de trabajo en seis zonas:

Pentagrama	Círculo	Triángulo
Todas (T)	Porygon	Cubo 3D

Estas divisiones son únicamente referencias lógicas utilizadas por el programa y no se dibujan sobre la hoja.

Tomando como referencia la hoja en la orientación utilizada durante la prueba final y considerando que HOME se encuentra en el extremo superior derecho, las figuras quedaron distribuidas físicamente de la siguiente manera: el triángulo en la zona superior derecha, el círculo en la zona superior central y el pentagrama en la zona superior izquierda. En la fila inferior se ubicaron el cubo 3D a la derecha, Porygon en el centro y el conjunto correspondiente al comando extttT a la izquierda.

En la calibración final se aumentó el tamaño de las figuras y también la separación entre posiciones. La separación lógica quedó aproximadamente en 660 intervalos entre columnas y 700 intervalos entre filas.

Cada comando individual parte desde HOME, se desplaza hasta la posición asignada, realiza el dibujo y luego vuelve a HOME. De esta forma cada figura mantiene aproximadamente la misma ubicación sobre la hoja.

VII-F. Figuras implementadas

VII-F1. Triángulo: El triángulo se realiza mediante desplazamientos diagonales y horizontales. Su tamaño fue aumentado durante la calibración para aprovechar mejor el espacio disponible.

VII-F2. Círculo: El círculo se aproxima mediante numerosos segmentos horizontales, verticales y diagonales de corta duración. Los tiempos fueron ajustados experimentalmente hasta obtener una forma más suave.

VII-F3. Pentagrama: El pentagrama se genera a partir de una matriz de movimientos que recorre las cinco puntas mediante segmentos cruzados.

Para el comando T se utiliza una copia reducida de la misma matriz del pentagrama original, conservando el orden y la geometría de los movimientos. Esto permitió obtener un resultado mejor que al utilizar una trayectoria diferente.

VII-F4. Cubo 3D: Como cuarta figura se implementó un cubo tridimensional simplificado. El recorrido combina movimientos horizontales, verticales y diagonales y se realiza prácticamente como un único trazo.

VII-F5. Porygon: El personaje asignado al Equipo 1 es **Porygon #137**. Para adaptarlo al plotter se utilizó una representación geométrica basada principalmente en segmentos rectos, incluyendo cabeza, cuerpo, patas y cola.

VII-G. Generación de movimientos

Las figuras se almacenan principalmente mediante matrices o tablas de movimientos. Cada elemento indica una dirección y una duración.

Las direcciones implementadas son:

- derecha e izquierda;
- arriba y abajo;
- diagonal superior derecha e izquierda;
- diagonal inferior derecha e izquierda.

También se utilizan comandos especiales para subir y bajar el trazador y para indicar el final de una trayectoria.

Para realizar movimientos diagonales se activan simultáneamente una salida horizontal y una vertical.

VII-H. Temporización

Timer1 genera una base temporal aproximada de:

$$T_{tick} = 10 \text{ ms}$$

Por lo tanto:

$$T_{mov} = N \cdot 10 \text{ ms}$$

donde N representa la cantidad de intervalos programados. Por ejemplo:

$$N = 120$$

produce aproximadamente:

$$T_{mov} = 120 \cdot 10 \text{ ms} = 1.2 \text{ s}$$

Los valores fueron ajustados experimentalmente debido a la respuesta mecánica del plotter, los relés y el PLC.

VII-I. Control del trazador

Los pines D2 y D3 generan las órdenes utilizadas para bajar y subir el trazador.

El programa mantiene un estado interno que indica si el trazador se encuentra arriba o abajo, evitando repetir innecesariamente el accionamiento del solenoide.

Se utiliza un pulso de accionamiento de aproximadamente 100 ms y un tiempo de estabilización posterior de aproximadamente 300 ms.

VII-J. Posición HOME y retorno automático

La posición HOME utilizada por el sistema se encuentra en el extremo superior derecho del área útil de dibujo. Esta referencia física explica que la distribución de las figuras se realice desde la derecha hacia la izquierda al observar la hoja en su orientación final.

Antes de iniciar una prueba se posiciona el cabezal en esta referencia. Como el sistema trabaja mediante temporización y no utiliza realimentación de posición en el programa, es importante comenzar siempre desde la misma posición física.

Después de cada dibujo individual el cabezal completa automáticamente el recorrido necesario para regresar a HOME. Esto evita que una nueva figura comience desde una posición incorrecta.

VII-K. Recuperación mediante EEPROM

Durante las pruebas se observó que el accionamiento de subida del solenoide podía provocar un reinicio del ATmega328P.

Inicialmente, este reinicio interrumpía el retorno del cabezal y el programa volvía directamente al menú, provocando errores en la posición de la siguiente figura.

Para resolver este problema se utilizó la memoria EEPROM. Antes del accionamiento que puede producir el reinicio se guarda una etapa de continuación.

Al reiniciar, el microcontrolador lee dicha etapa y completa los movimientos que habían quedado pendientes hasta regresar a HOME.

Cuando el retorno termina correctamente, la etapa almacenada se borra y el programa vuelve al menú.

Este mecanismo se aplica tanto a los comandos individuales como al comando T.

VII-L. Comando T: todas las figuras

El comando T desplaza el cabezal hasta la zona inferior izquierda de la hoja y dibuja allí versiones reducidas de las cinco figuras.

La distribución interna es aproximadamente:

Pentagrama	Círculo	Triángulo
Porygon	Cubo 3D	

Durante las últimas pruebas se aumentó ligeramente el tamaño de estas figuras para que pudieran distinguirse mejor. En el resultado experimental final se observan las cinco figuras reducidas agrupadas en la zona inferior izquierda, separadas del resto de los dibujos individuales.

El triángulo, círculo y Porygon utilizan matrices derivadas de las figuras originales con una escala aproximada del 62.5 %. El pentagrama utiliza una copia de la matriz original a una escala aproximada del 81.25 %, mientras que el cubo también fue aumentado respecto de las primeras pruebas.

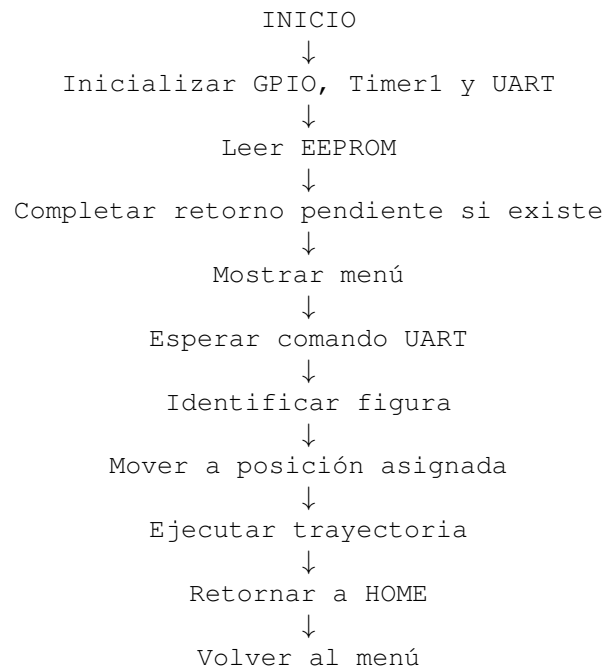
Al finalizar las cinco figuras, el cabezal regresa nuevamente a HOME.

VII-M. Secuencia general de funcionamiento

1. Posicionar el cabezal en HOME, ubicado en el extremo superior derecho.
2. Compilar el código en Microchip Studio.
3. Generar y cargar el archivo hexadecimal en el Arduino UNO.
4. Habilitar la estación del plotter.
5. Abrir el terminal serial a 9600 baud.
6. Seleccionar 1, 2, 3, 4, P o T.
7. Desplazar el cabezal hasta la posición asignada.
8. Ejecutar automáticamente la trayectoria.
9. Levantar el trazador.
10. Completar el retorno a HOME.
11. Mostrar nuevamente el menú.

VII-N. Diagrama de funcionamiento

La lógica general puede resumirse como:



VII-Ñ. Código ensamblador

El programa final incluye:

- inicialización del Stack Pointer;
- configuración de GPIO;
- comunicación UART;
- Timer1;
- control del solenoide;
- rutinas de movimiento;
- interpretación de matrices;

- posicionamiento de las figuras;
- retorno automático a HOME;
- recuperación mediante EEPROM;
- ejecución individual y mediante el comando T.

Acceso al código ASM del plotter en GitHub

Figura 8. Enlace al repositorio del código fuente del Problema C.

VII-O. Resultados experimentales

Durante las pruebas físicas se realizaron sucesivos ajustes hasta obtener una distribución adecuada en la hoja A4.

Los principales cambios realizados fueron:

- aumento del tamaño de las figuras individuales;
- aumento de la separación entre posiciones;
- asignación de una posición fija a cada figura;
- implementación del cubo 3D;
- ajuste del círculo y del pentagrama;
- implementación de Porygon;
- incorporación de las cinco figuras en el comando T;
- aumento final del tamaño de las figuras del comando T;
- retorno automático a HOME;
- recuperación después de reinicios mediante EEPROM.

La fotografía final confirmó que la posición física de los dibujos no sigue el orden numérico de izquierda a derecha. Debido a que HOME está ubicado en el extremo superior derecho, la disposición observada sobre la hoja fue: pentagrama–círculo–triángulo en la fila superior y todas–Porygon–cubo 3D en la fila inferior.

La distribución final es:

3 – Pentagrama	2 – Círculo	1 – Triángulo
T – Todas	P – Porygon	4 – Cubo 3D

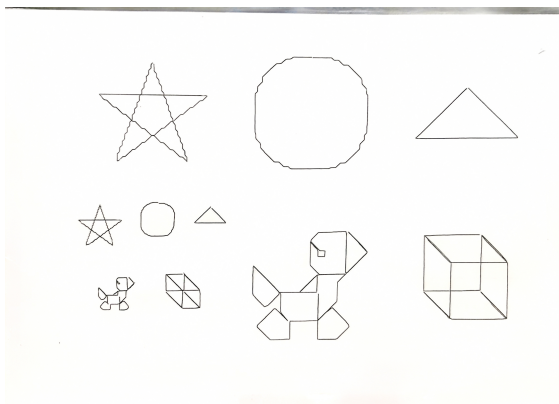


Figura 9. Resultado experimental obtenido con el plotter.

Acceso directo al video demostrativo en Google Drive

VII-P. Conclusión del Problema C

Se logró controlar el plotter mediante el ATmega328P utilizando lenguaje ensamblador y comunicación UART.

El sistema permite seleccionar diferentes figuras desde una computadora, ejecutar automáticamente sus trayectorias y distribuirlas de forma ordenada sobre una hoja A4.

Las pruebas demostraron la importancia de calibrar los tiempos de movimiento, especialmente en figuras formadas por numerosos segmentos cortos.

La incorporación del retorno a HOME y del almacenamiento de etapas en EEPROM permitió mejorar la repetibilidad del sistema frente a los reinicios observados durante el accionamiento del solenoide.

Finalmente, se obtuvo una versión capaz de dibujar individualmente las cinco figuras y de representarlas simultáneamente mediante el comando T.

VIII. PROBLEMA D: COMUNICACIÓN USART–UART ENTRE DOS ATMEGA328P

VIII-A. Descripción

En este problema se implementó una comunicación USART/UART entre dos microcontroladores ATmega328P montados sobre placas Arduino UNO.

El primer microcontrolador funciona como transmisor y utiliza un DIP switch de tres posiciones conectado a las entradas PC0, PC1 y PC2. Estas entradas permiten seleccionar un valor binario de tres bits, comprendido entre 000 y 111, equivalente a los valores decimales entre 0 y 7.

El valor seleccionado es transmitido mediante USART al segundo microcontrolador. El receptor interpreta el dato recibido y activa uno de ocho LEDs de acuerdo con el valor correspondiente.

Por ejemplo, para la combinación:

$$011 = 3$$

el receptor activa únicamente el LED asociado al valor decimal 3.

VIII-B. Estructura del sistema

El funcionamiento general del sistema se divide en las siguientes etapas:

1. Leer las tres posiciones del DIP switch en el microcontrolador transmisor.
2. Invertir los niveles de entrada debido al uso de las resistencias *pull-up* internas.
3. Construir un valor binario de tres bits entre 000 y 111.
4. Transmitir el valor mediante USART.
5. Recibir el byte en el segundo ATmega328P.
6. Decodificar el valor recibido.
7. Apagar los LEDs previamente activos.
8. Encender únicamente el LED correspondiente al valor recibido.

VIII-C. Entradas del transmisor

Para la selección del valor binario se utilizó un DIP switch de tres posiciones. Las entradas fueron conectadas de acuerdo con la Tabla VIII.

Cuadro VIII
ASIGNACIÓN DE ENTRADAS DEL DIP SWITCH

Bit	ATmega328P	Arduino UNO
Bit 0	PC0	A0
Bit 1	PC1	A1
Bit 2	PC2	A2

Se utilizaron las resistencias *pull-up* internas del microcontrolador. Por esta razón, eléctricamente una entrada conectada a GND presenta un nivel lógico bajo. En el programa se invierte posteriormente la lectura mediante la instrucción:

```
1 com dato
2 andi dato, 0x07
```

De esta manera, una posición activada del DIP switch representa un 1 lógico y una posición desactivada representa un 0 lógico.

VIII-D. Conexión USART

La comunicación se realizó en un único sentido, desde el transmisor hacia el receptor. Por lo tanto, únicamente fue necesario conectar la salida TX del primer microcontrolador con la entrada RX del segundo, además de compartir la referencia de tierra.

Arduino 1 D1/TX → Arduino 2 D0/RX
Arduino 1 GND ↔ Arduino 2 GND

No fue necesario conectar la salida TX del segundo microcontrolador, debido a que el receptor no transmite información de regreso.

Los parámetros utilizados para la comunicación se presentan en la Tabla IX.

Cuadro IX
PARÁMETROS DE COMUNICACIÓN USART

Parámetro	Valor
Frecuencia de reloj	16 MHz
Baud rate	9600 baudios
Bits de datos	8 bits
Paridad	Sin paridad
Bits de parada	1 bit
Configuración	8N1
UBRR0	103

Para una frecuencia de reloj de 16 MHz y una velocidad de 9600 baudios se utilizó:

$$UBRR0 = 103$$

VIII-E. Salidas del receptor

El segundo microcontrolador dispone de ocho LEDs. Cada uno representa uno de los posibles valores recibidos entre 0 y 7.

La distribución utilizada se muestra en la Tabla X.

Cuadro X
ASIGNACIÓN DE LOS LEDs DEL RECEPTOR

LED	ATmega328P	Arduino UNO
LED 0	PB0	D8
LED 1	PB1	D9
LED 2	PB2	D10
LED 3	PB3	D11
LED 4	PB4	D12
LED 5	PB5	D13
LED 6	PC0	A0
LED 7	PC1	A1

VIII-F. Cálculo de las resistencias de los LEDs

Para limitar la corriente que circula por cada uno de los LEDs del receptor se utilizó una resistencia en serie.

Aplicando la Ley de Ohm:

$$I_{LED} = \frac{V_{CC} - V_{LED}}{R}$$

Considerando una tensión de alimentación de:

$$V_{CC} = 5V$$

una caída de tensión aproximada en el LED de:

$$V_{LED} \approx 2V$$

y una resistencia utilizada experimentalmente de:

$$R = 220\Omega$$

se obtiene:

$$I_{LED} = \frac{5 - 2}{220}$$

$$I_{LED} = 0.0136A$$

Por lo tanto:

$$I_{LED} \approx 13.6mA$$

De esta manera, cada uno de los ocho LEDs del microcontrolador receptor fue conectado en serie con una resistencia de 220Ω , limitando la corriente a aproximadamente $13.6mA$.

VIII-G. Tabla de funcionamiento

La correspondencia entre el valor seleccionado en el DIP switch y el LED activado se presenta en la Tabla XI.

Cuadro XI
CORRESPONDENCIA ENTRE ENTRADA BINARIA Y LED ACTIVADO

Entrada binaria	Decimal	LED activado
000	0	LED 0 – D8
001	1	LED 1 – D9
010	2	LED 2 – D10
011	3	LED 3 – D11
100	4	LED 4 – D12
101	5	LED 5 – D13
110	6	LED 6 – A0
111	7	LED 7 – A1

VIII-H. Código del transmisor

El programa del transmisor configura PC0, PC1 y PC2 como entradas con resistencias *pull-up* internas. La lectura se invierte y se enmascaran únicamente los tres bits menos significativos.

El dato es transmitido solamente cuando se detecta un cambio en el valor seleccionado.

[Acceso al código ASM del MICRO1 – Transmisor en GitHub](#)

Figura 10. Código del transmisor.

VIII-I. Código del receptor

El receptor permanece esperando la llegada de un byte mediante USART. Una vez recibido, conserva únicamente los tres bits menos significativos, obteniendo un valor comprendido entre 0 y 7.

Antes de activar una nueva salida se apagan todos los LEDs y posteriormente se habilita únicamente el correspondiente al dato recibido.

[Acceso al código ASM del MICRO2 – Receptor en GitHub](#)

Figura 11. Enlace al código fuente del microcontrolador receptor del Problema D.

VIII-J. Simulación

[Acceso a la simulación y captura de pantalla del circuito Problema D en GitHub](#)

VIII-K. Resultados y evidencia

Durante la verificación experimental se comprobó inicialmente el montaje de los ocho LEDs mediante una rutina de prueba que los activaba de forma secuencial. De esta manera se verificó que las conexiones entre los puertos PB0–PB5 y PC0–PC1 y sus respectivos LEDs eran correctas.

Posteriormente se cargaron los programas definitivos de transmisión y recepción. Al modificar la combinación del DIP switch en el primer microcontrolador, el valor correspondiente fue enviado por USART al segundo ATmega328P.

El receptor respondió activando únicamente el LED asociado al valor recibido. Por ejemplo, para la entrada 011, correspondiente al valor decimal 3, se verificó la activación del LED conectado a D11.

Las pruebas realizadas confirmaron el funcionamiento correcto de la comunicación USART entre ambos microcontroladores y la correcta decodificación de los valores binarios comprendidos entre 000 y 111.

[Acceso directo a la evidencia experimental en Google Drive](#)

Figura 12. Montaje experimental de la comunicación USART entre los dos ATmega328P.

VIII-L. Video demostrativo

La verificación de las combinaciones binarias 000–111 y la respuesta de los LEDs del receptor se documenta mediante el siguiente video.

[Acceso directo al video demostrativo en Google Drive](#)

VIII-M. Conclusión

Se implementó correctamente una comunicación USART entre dos microcontroladores ATmega328P. El transmisor permitió seleccionar ocho valores diferentes mediante un DIP switch de tres bits, mientras que el receptor decodificó el dato recibido y activó el LED correspondiente.

La implementación permitió comprobar experimentalmente el funcionamiento de una comunicación UART asíncrona configurada a 9600 baudios, 8 bits de datos, sin paridad y un bit de parada.

IX. PROBLEMA E: AUTOMATIZACIÓN DE UNA PUERTA DE GARAJE

IX-A. Descripción

El Problema E consiste en programar un ATmega328P para controlar una puerta de garaje mediante una máquina de estados. El sistema posee dos pulsadores de control remoto, uno para solicitar apertura y otro para solicitar cierre, dos sensores de posición y un sensor de obstáculos.

El movimiento de la puerta se realiza mediante dos salidas de control del motor: una para apertura y otra para cierre. Durante el movimiento se activa una alarma sonora y se indica visualmente el estado mediante LEDs. El sistema informa además su estado mediante USART, lo que permite verificar su funcionamiento desde una terminal serial.

IX-B. Entradas y salidas

Cuadro XII
ENTRADAS Y SALIDAS DE LA PUERTA DE GARAJE

Tipo	Elemento	Función
Entrada	Pulsador abrir (PD2)	Solicitar apertura
Entrada	Pulsador cerrar (PD3)	Solicitar cierre
Entrada	Sensor S1 – puerta abierta (PD4)	Detectar posición superior
Entrada	Sensor S2 – puerta cerrada (PD5)	Detectar posición inferior
Entrada	Sensor de obstáculo (PB0)	Detener el movimiento por seguridad
Salida	Motor abrir (PB1)	Accionar apertura
Salida	Motor cerrar (PB2)	Accionar cierre
Salida	Alarma sonora – buzzer pasivo (PB3)	Advertir movimiento
Salida	LEDs indicadores (PC0–PC3)	Mostrar el estado actual
Salida	USART (PD1/TXD)	Informar estado del sistema

Todas las entradas se configuraron utilizando las resistencias *pull-up* internas del ATmega328P, por lo que eléctricamente un pulsador presionado o un sensor activado se traduce en un nivel bajo. Esta lectura se invierte por software para trabajar internamente con lógica positiva.

El buzzer utilizado es de tipo **pasivo**: no posee electrónica propia para generar el tono, por lo que un nivel fijo en su entrada no produce ningún sonido. Esto se tuvo en cuenta en el diseño de la salida de alarma, como se detalla en la Sección IX-G.

IX-C. Máquina de estados

La solución se organiza mediante una máquina de estados finita de cinco estados:

- **CERRADA**: el motor permanece detenido y el sistema espera una solicitud de apertura.
- **ABRIENDO**: se activa la salida de apertura, se enciende el LED correspondiente y suena la alarma mientras la puerta se desplaza.
- **ABIERTA**: el motor se detiene al detectarse el sensor S1 y el sistema espera una solicitud de cierre.
- **CERRANDO**: se activa la salida de cierre, se enciende el LED correspondiente y suena la alarma durante el desplazamiento.
- **DETENIDA**: estado de seguridad en el cual el motor y la alarma se apagan y el sistema queda a la espera de una nueva orden.

Las condiciones de transición se resumen en la Tabla XIII.

Cuadro XIII
TRANSICIONES DE LA MÁQUINA DE ESTADOS

Estado actual	Condición	Estado siguiente
CERRADA	Pulsador abrir	ABRIENDO
ABRIENDO	Sensor S1 (puerta abierta)	ABIERTA
ABIERTA	Pulsador cerrar	CERRANDO
CERRANDO	Sensor S2 (puerta cerrada)	CERRADA
ABRIENDO	Pulsador cerrar (sentido contrario)	DETENIDA
CERRANDO	Pulsador abrir (sentido contrario)	DETENIDA
DETENIDA	Pulsador abrir	ABRIENDO
DETENIDA	Pulsador cerrar	CERRANDO
Cualquier estado	Obstáculo detectado	DETENIDA

Si bien el enunciado original describe la detención por obstáculo mientras la puerta se encuentra en movimiento, en la implementación final se otorgó prioridad al sensor de obstáculo sobre los demás eventos. De esta manera, cuando se detecta un obstáculo el sistema pasa al estado DETENIDA y desactiva las salidas asociadas al movimiento.

Si el sistema ya se encuentra en el estado DETENIDA, el mensaje de seguridad no se retransmite continuamente para evitar saturar la comunicación USART.

IX-D. Detección del obstáculo mediante polling

El enunciado original solicita que el sensor de obstáculos se conecte a una fuente de interrupción distinta de INT0 e INT1. Por este motivo, durante el desarrollo se evaluó inicialmente el uso de una *Pin Change Interrupt* sobre el pin PB0.

La primera implementación configuraba los registros PCICR y PCMSK0 y utilizaba una rutina de atención asociada al cambio de nivel del sensor.

Durante las pruebas realizadas en Proteus 8 Professional no se obtuvo la respuesta esperada de la interrupción al modificar el estado del pin. Para analizar el comportamiento se realizaron pruebas de diagnóstico utilizando señales de salida y mensajes USART.

Posteriormente se comprobó que la lectura directa del mismo pin PB0 mediante *polling* detectaba correctamente el cambio de estado del sensor en la simulación utilizada.

Por este motivo, en la versión final implementada y ensayada el sensor de obstáculos se consulta mediante *polling* dentro del ciclo principal. En cada iteración se verifica el estado de PB0 y se detecta una nueva activación del sensor. Cuando esto ocurre se establece una bandera que es evaluada con prioridad por la máquina de estados.

La solución mediante polling permitió obtener experimentalmente la detención inmediata del movimiento durante las pruebas realizadas. Sin embargo, se deja expresamente documentado que esta implementación no utiliza una interrupción de hardware para el sensor de obstáculos y constituye una modificación respecto al requisito original del enunciado.

IX-E. Mensajes USART

El sistema transmite, ante cada cambio de estado, uno de los siguientes mensajes:

- Puerta cerrada
- Puerta abriendo
- Puerta abierta
- Puerta cerrando
- Obstaculo detectado
- Movimiento detenido por seguridad

La comunicación se configuró a 9600 baudios, 8 bits de datos, sin paridad y un bit de parada (8N1). Para una frecuencia de reloj de 16 MHz se utilizó:

$$UBRR0 = 103$$

IX-F. Diagrama de estados

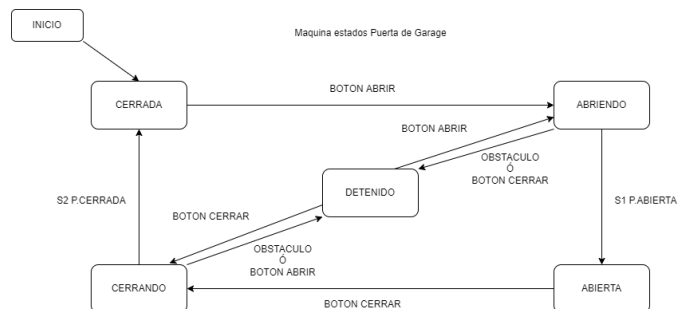


Figura 13. Máquina de estados de la puerta de garaje.

IX-G. Explicación de la solución programada

El programa se estructura alrededor de una variable de estado que determina qué rutina se ejecuta en cada vuelta del ciclo principal. En cada iteración se leen los pulsadores y los sensores de posición, se consulta el sensor de obstáculo mediante polling, se ejecuta la máquina de estados y se actualizan las salidas correspondientes.

Para evitar que mantener presionado un pulsador produzca repetidamente la misma orden, el programa conserva el estado anterior de las entradas y detecta únicamente nuevas activaciones mediante flancos.

La inicialización configura el Stack Pointer, los puertos de entrada y salida con sus resistencias *pull-up* y el módulo

USART. En la versión final implementada, el control de las entradas se realiza mediante sondeo dentro del bucle principal.

El sensor de obstáculo posee prioridad dentro de la máquina de estados. Cuando se detecta una nueva activación, se establece una bandera de seguridad, se cambia al estado DETENIDA y se desactivan las salidas de movimiento.

Como el buzzer utilizado es pasivo, necesita una señal variable para producir un sonido audible. Por este motivo, durante los estados ABRIENDO y CERRANDO se genera la señal necesaria para accionar la alarma, mientras que en los demás estados permanece apagada.

Cada subrutina de salidas (SALIDAS_CERRADA, SALIDAS_ABRIENDO, SALIDAS_ABIERTA, SALIDAS_CERRANDO y SALIDAS_DETENIDA) actualiza los LEDs y las salidas del motor correspondientes al estado actual.

El USART se utiliza además como recurso de diagnóstico, transmitiendo un mensaje ante cada cambio de estado. Esto permitió comprobar el comportamiento del sistema tanto durante la simulación como durante las pruebas realizadas sobre la placa física.

IX-H. Código ensamblador

[Acceso al código ASM de la puerta de garaje en GitHub](#)

Figura 14. Enlace al código fuente del Problema E.

IX-I. Simulación

[Acceso a la simulación y la captura de pantalla del circuito Problema E en GitHub](#)

IX-J. Resultados y evidencia

El sistema fue compilado en Microchip Studio y programado sobre una placa Arduino UNO física mediante avrdude, verificándose además previamente en Proteus 8 Professional.

Se comprobó el ciclo completo de apertura y cierre. Al presionar el pulsador de abrir con la puerta cerrada, se activa la salida correspondiente al movimiento de apertura, se enciende el LED de ABRIENDO y se activa la alarma. Al alcanzar el sensor S1, el movimiento se detiene y se indica el estado de puerta abierta.

El ciclo de cierre se comprobó de manera equivalente. Al solicitar el cierre se activa la salida correspondiente hasta que el sensor S2 indica que la puerta alcanzó la posición cerrada.

También se verificó que, al presionar el pulsador correspondiente al sentido contrario mientras la puerta se encuentra en movimiento, el sistema pasa al estado DETENIDA. Desde este estado puede recibirse posteriormente una nueva orden de apertura o cierre.

La detección de obstáculos se comprobó mediante el pulsador conectado a PB0. Al activarlo, el sistema pasa al estado DETENIDA, desactiva el movimiento e informa mediante USART:

Obstaculo detectado

Movimiento detenido por seguridad

Como se explicó en la Sección IX-D, en la versión final esta entrada se consulta mediante polling en lugar de utilizar una interrupción de hardware.

Los mensajes de estado fueron verificados tanto mediante la Terminal Virtual de Proteus como, sobre la placa física, mediante el Data Visualizer de Microchip Studio conectado al puerto serie a 9600 baudios.

IX-K. Video demostrativo

[Acceso directo al video demostrativo en Google Drive](#)

X. GESTIÓN DEL CÓDIGO CON GITHUB

El código fuente de las distintas actividades se gestiona mediante GitHub. El repositorio permite mantener disponibles las versiones de los programas y registrar las modificaciones significativas realizadas durante el desarrollo.

[Acceso al repositorio GitHub de Tecnologías de Microprocesamiento](#)

XI. RESULTADOS GENERALES

El Laboratorio 2 integra distintos recursos del ATmega328P en cinco aplicaciones. La matriz de LEDs combina multiplexado y selección mediante UART; el DAC R-2R incorpora tablas LUT, temporización y generación de señales analógicas; el plotter utiliza secuencias de movimiento y comunicación serial; la comunicación entre dos ATmega328P permite transmitir y decodificar datos; y la puerta de garaje combina máquina de estados, sensores, lectura por polling, salidas de control y mensajes USART.

Los resultados experimentales se incorporan en las subsecciones correspondientes. El Problema D fue verificado físicamente para las combinaciones binarias comprendidas entre 000 y 111. Los resultados experimentales se incorporan en las subsecciones correspondientes, junto con las evidencias, simulaciones y videos demostrativos disponibles para cada problema.

XII. CONCLUSIONES

El Laboratorio 2 permite ampliar el manejo de la arquitectura AVR más allá del control básico de GPIO. Las actividades propuestas requieren integrar memoria, temporización, interrupciones, tablas de consulta y comunicación serie dentro de programas desarrollados completamente en lenguaje ensamblador.

La utilización de LUT y temporizadores permite generar secuencias con tiempos controlados, mientras que USART/UART posibilita la interacción con el usuario y la comunicación entre dispositivos. Por otra parte, las máquinas de estados resultan adecuadas para organizar sistemas secuenciales como el plotter y la puerta de garaje.

La simulación previa, la verificación de las conexiones y el registro de las modificaciones en GitHub forman parte del proceso de desarrollo y permiten mantener trazabilidad entre las versiones iniciales y las implementaciones finales de cada problema.

REFERENCIAS

- [1] Microchip Technology Inc., *ATmega328P 8-bit AVR Microcontroller Datasheet*.
- [2] UTEC, Ingeniería Mecatrónica, *Laboratorio 2 – Tecnologías de Microprocesamiento*.